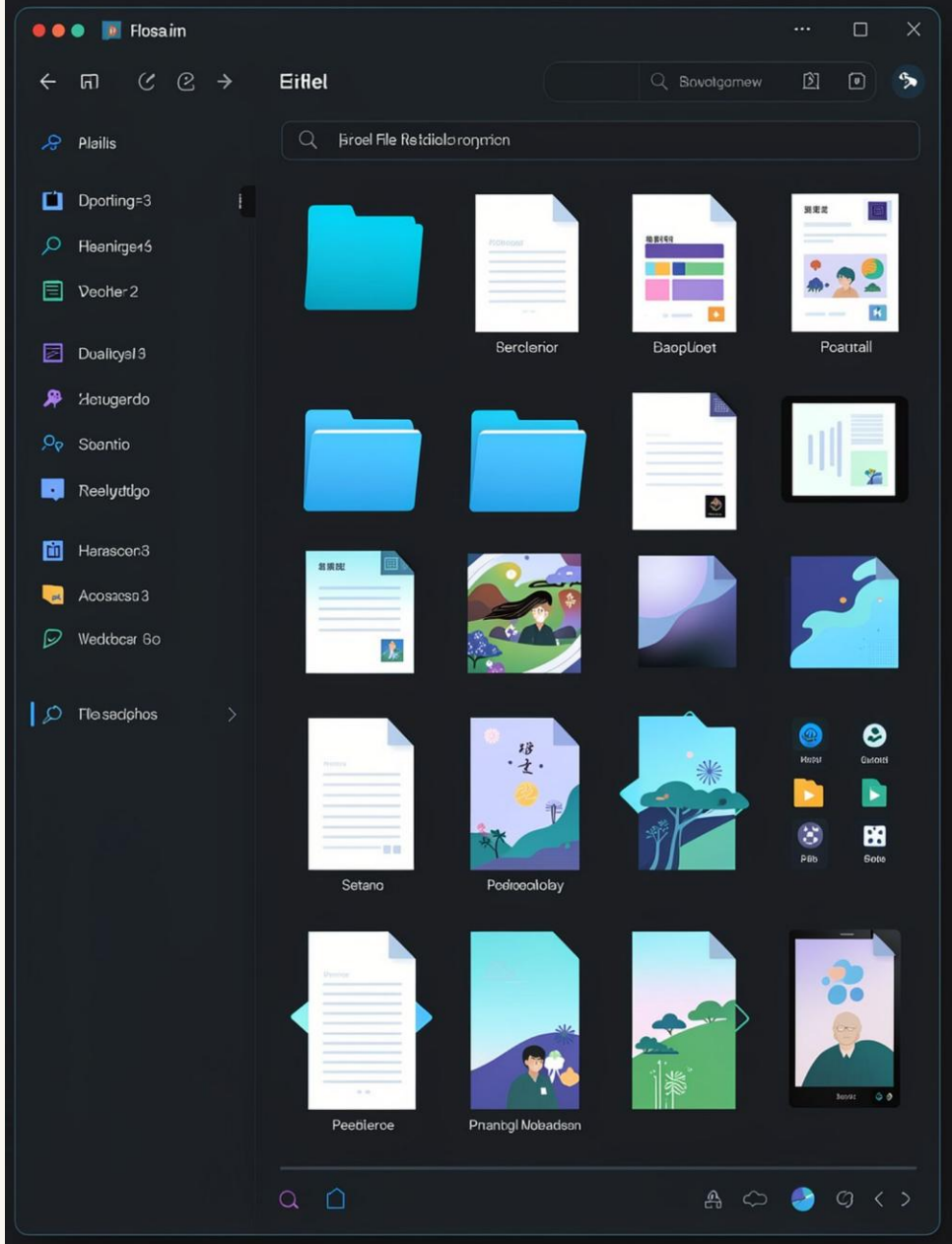
SMART FILE & MEDIA MANAGER

- Advanced Programming — Assessment 2
- Task 15: Technical documentation & demonstration

The Problem and Solution

- One interface for folder scanning and browsing
- Search, filter, preview images, playback MP3 files, and export to CSV
- Background processing ensures responsive interface



System Structure

01

main.py is the application entry point

02

main_window.py handles the interface and user interaction

03

controller.py is the application controller

04

file_model.py handles scanning and file processing

05

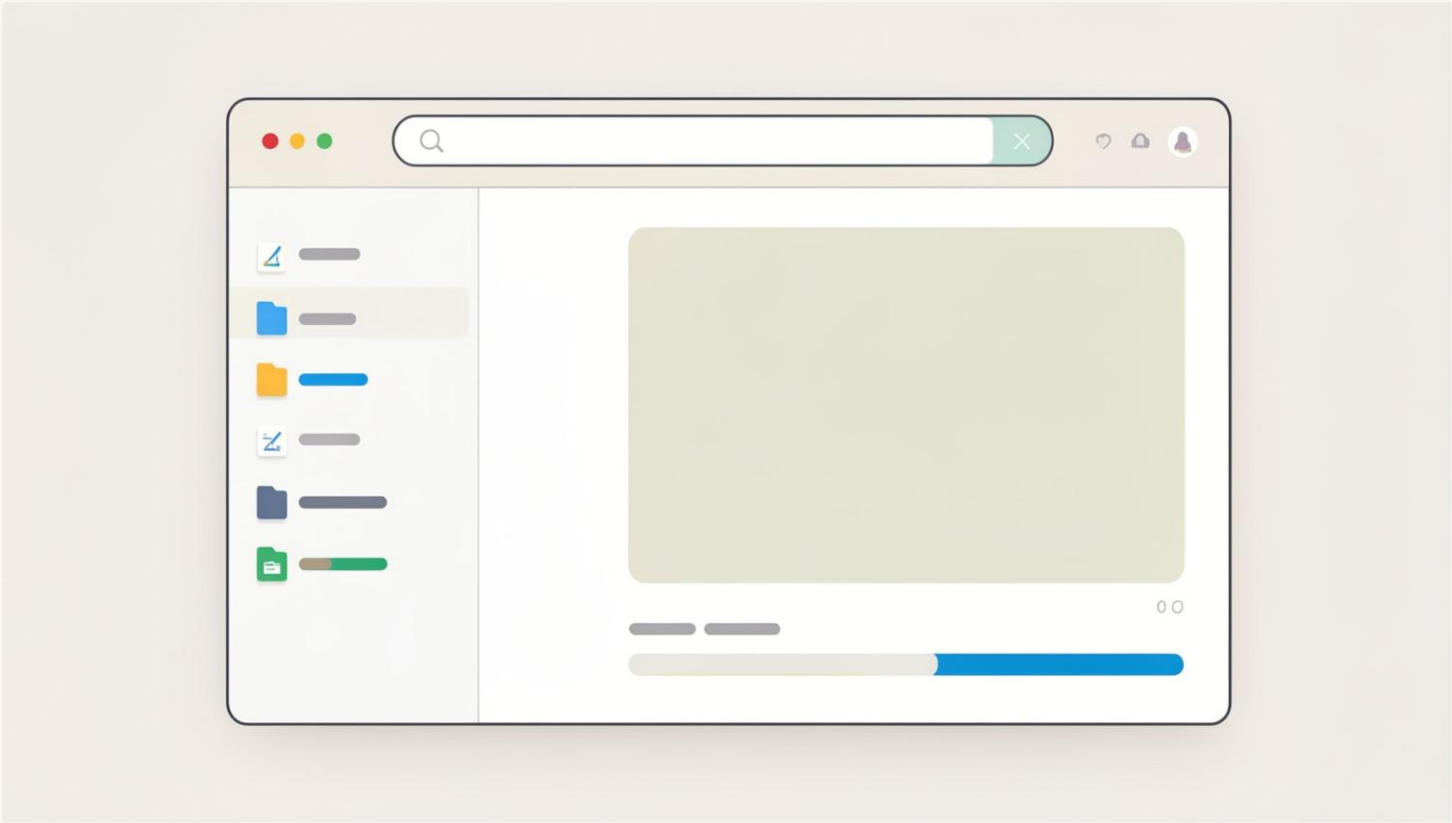
workers.py handles background processes

Interface Design

Folder/scan/cancel/export actions

Search and type filters

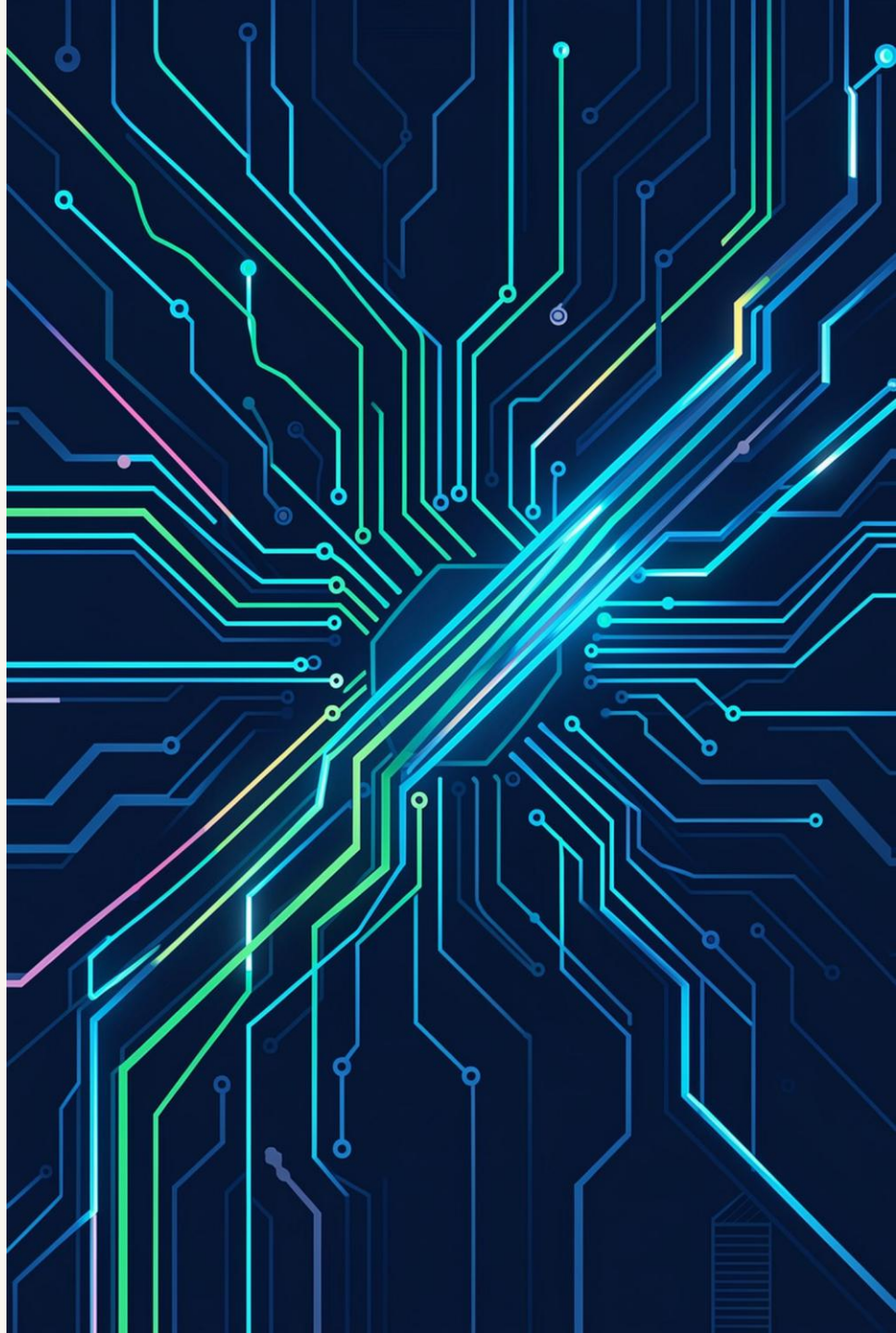
Status and progress feedback



Demonstrate scan, search, preview, and export features

Concurrency and Safety

- Folder scanning happens off the GUI event loop
- Workers send progress/results via Qt signals
- UI updates happen on the GUI thread
- Cancellation is done in a cooperative manner



Performance — Actual vs Theoretical

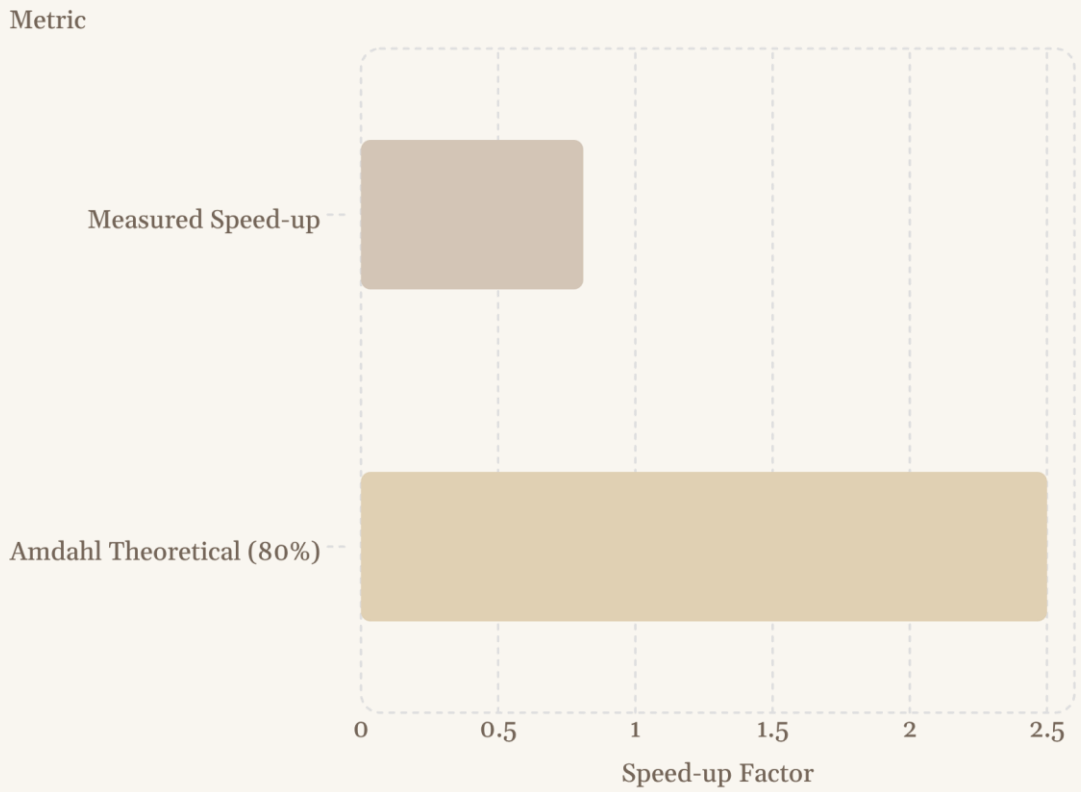
350 files • 4 workers • 3 runs

Sequential avg: 0.0181 seconds

Threaded avg: 0.0224 seconds

Measured speed-up: 0.81× (slower)

Amdahl theoretical result (80% parallel): 2.50×



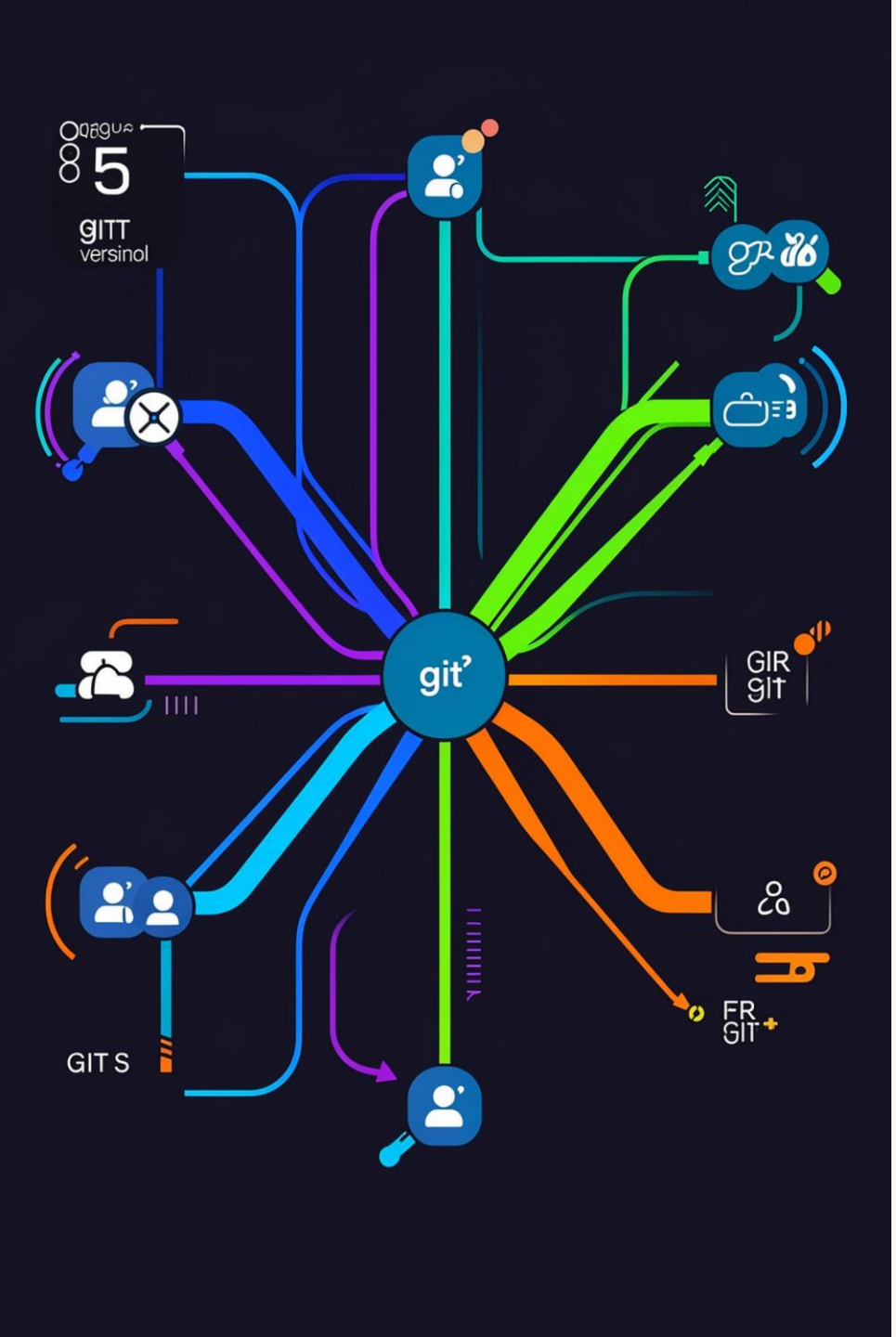
Testing and Debugging

10 checks done manually, reported passed

6 unit tests, passed in (0.043 seconds)

8 packaged EXE checks passed

Test discovery issue was solved by using tests/ and explicit discovery



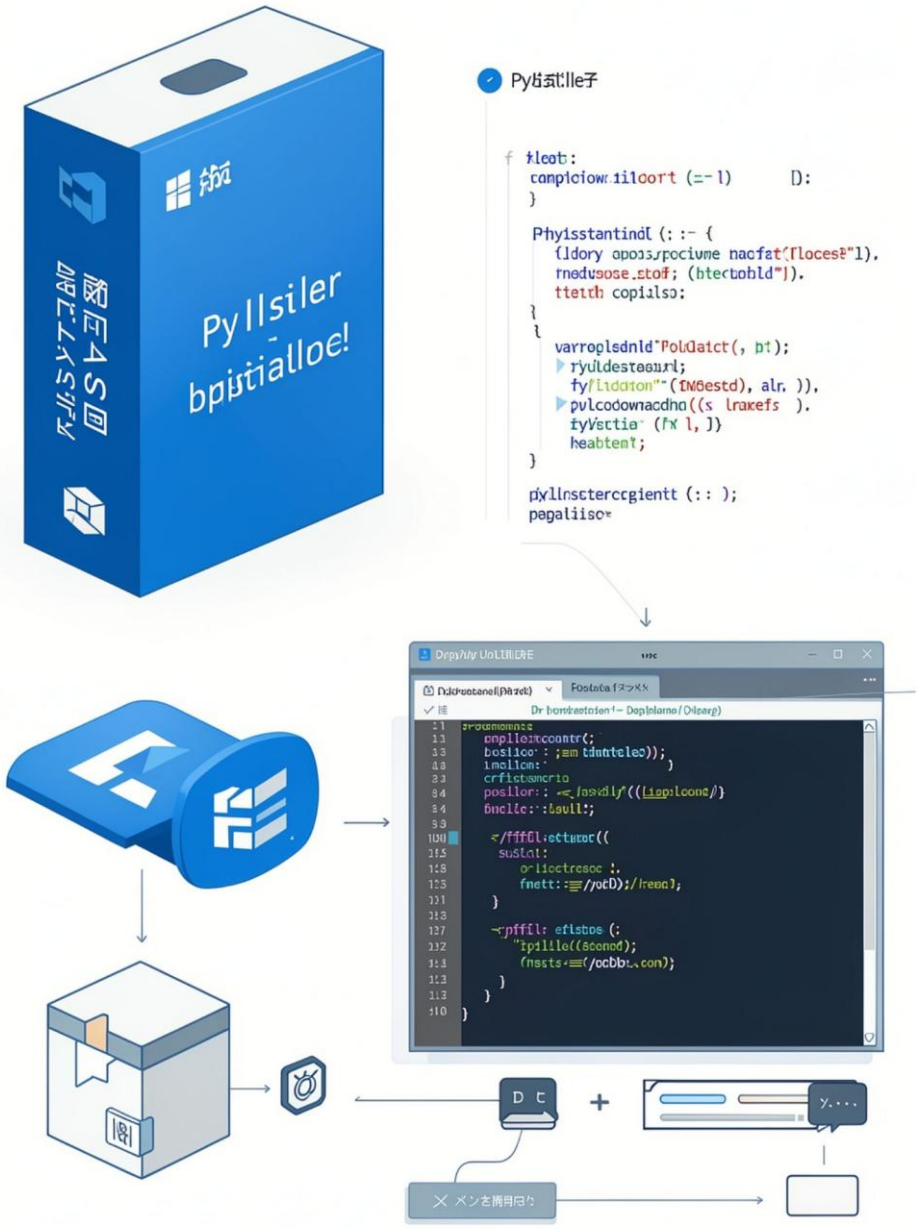
Git Workflow

1. Feature branch with meaningful commits
2. GitHub Pull Request #1 merged to main
3. Controlled merge conflict, resolved on both branches
4. Distribution Commit: 6032498

Windows Distribution

Installation of a full build using PyInstaller v.6.22.3 (one-folder application) with the latest version of Python (Python 3.13), plus an application that was run and successfully completed all eight of our local tests

Ready to be packaged in a zip file along with the **Distribution.md** and .spec files for further redistribution.



Evaluation & Next Steps

The strengths are as follows:

- A user interface that can be used.
- Responsive scans.
- Passed all tests.
- The build is repeatable.

The limits we identified were as follows:

- Small size of the performance sample we tested it with.
- Threaded operations will perform at a slightly lower rate than other methods of performing this operation.

Further work includes:

- Larger samples for testing performance.
- GUI automation.
- Distribution testing from another PC.



Live Demo

1. Pick a folder → Scan for media → Search and filter results
2. View image preview → Play MP3 → Export CSV → Cancel/Restart
3. View tests → View performance → View pull request history → Execute